Q1.1 Selected problem

1 Point

Grading comment:
(3-4 sentences): Identify the users, the task they need to accomplish, and the limitation in the current approaches. Explain how software could improve the outcome and connect the problem to specific regulatory obligations. Cite the relevant provisions.

Manufacturers of products with digital elements, particularly product security engineers and compliance personnel, need to determine which supported product releases are affected by newly discovered vulnerabilities and manage the evidence and reporting that follows. Today they scan the active development branch and track supported versions, incident records, and reporting deadlines across separate tools and spreadsheets. Because the SBOM obligation is attached per release placed on the market rather than per codebase, older releases still within their support period often have no retrievable component list, so the affected-release question cannot be answered reliably or within the time the reporting deadlines allow. Software could bind component records to specific shipped releases, organize investigation evidence, and track reporting deadlines while leaving final engineering and compliance decisions to human personnel — supporting the CRA requirement to identify and document vulnerabilities and components through a machine-readable SBOM under Annex I, Part II(1), and the Article 14(1)–(2) requirement to report actively exploited vulnerabilities within 24 and 72 hours with a final report after corrective measures become available.

the sources for each claim, all pointing to the official text on EUR-Lex:
https://eur-lex.europa.eu/eli/reg/2024/2847/oj

Q1.2 Selection rationale

1 Point

Grading comment:
(3-5 sentences): Name the Phase 1 alternatives considered and explain why the pod selected or combined these ideas. Explain why the problem involves meaningful engineering work, fits the semester, and permits independent implementations whose outputs can be compared.

The pod considered four related Phase 1 concepts. Team 6 proposed ReleaseGate, a release-indexed SBOM and vulnerability-matching system, and IncidentClock, a deadline-tracking and audit system for Article 14 reporting, while team 2 independently proposed the Automated SBOM and Vulnerability Tracking System and the CRA Article 14 Incident Reporting System, which split the same two problems the same way and both planned to reuse OWASP Dependency-Track for vulnerability matching. Because the two teams reached nearly identical problem decompositions, the pod combined the ideas into one shared specification instead of adopting either team's version outright, leaving the remaining differences, such as binding SBOMs to specific shipped releases versus generating them live in CI/CD, and using temporal engine versus TheHive for deadline tracking, as implementation choices for each team's individual design in Phase 7. The combined problem involves real engineering work, including matching vulnerabilities to components and releases, maintaining persistent incident state, computing regulatory deadlines, and producing consistent report data, while staying small enough for a semester by reusing existing vulnerability-matching and workflow tools. It also supports independent implementations, since each team can choose its own architecture while being tested against the same inputs and compared on the same observable outputs: affected releases, incident status, deadlines, and generated report data.

Q1.3 Pod agreement:

1 Point

Grading comment:

Record the meeting date, participating teams and representatives, and the agreed specification version or date. State any remaining disagreement that affects the common scope or testing behavior and how the pod will resolve it. If no disagreement remains, state that.

Meeting date: September 26, 2026

Participating teams: Team 2 and Team 6

Representatives: Chris Feng, Pataraphol Keeraswangporn, Jared Dong, Ansh Kothari, Sathvik Valluri, Daksh Agarwal, Donovan Baran, and Sanjay Sriram

Agreed specification: Common specification v1.0, agreed September 24, 2026. It covers REQ-01 through REQ-08, acceptance criteria AC-01 through AC-10, and the Section 5 web-service interface.

Remaining disagreements: One disagreement affects common testing behavior: whether Article 14 "awareness" starts at the raw alert time or at engineer-verified confirmation. The pod resolved this for testing by treating awareness_time as a user-supplied field (REQ-05). The system never infers awareness, so both teams produce identical deadlines from the same inputs. Any legal interpretation is outside the common scope and is tracked as Risk 2 in Section 9.

Two other differences remain, but neither affects common scope or testing, because both teams are tested only through the Section 5 interface:

Whether to bind SBOMs to shipped releases or generate them live in CI/CD.

Whether to track deadlines with a workflow engine (Temporal or TheHive) or compute them from stored timestamps.

Each team will decide these in its own design.

Q2.1 System boundary

1 Point

Grading comment:
Insert a context or boundary diagram showing the common system, users, external systems, and the main information exchanged across the boundary. Label responsibilities retained by people or external services. Use any clear notation; identify assumptions the diagram relies on.

We have the diagrams and just updating them to the needs

2.1.svg diagram

Included:

1. Release registration with a support-end date (REQ-01; Art. 13(8))
2. CycloneDX JSON SBOM upload, validation and rejection with a stated reason (REQ-02; Annex I, Part II(1))
3. CVE lookup grouped by support status, with an explicit message when the CVE is unknown or the source is unreachable (REQ-03, REQ-04; Annex I, Part II(1)–(2))
4. Incident cases with awareness time and classification, plus tracking of the 24h, 72h and 14-day deadlines (REQ-05, REQ-06; Art. 14(1)–(2), Art. 15)
5. Report data for each stage as JSON, and an append-only audit log (REQ-07, REQ-08; Art. 14(2), Art. 16)

Excluded:

1. Generating or attesting SBOMs (done by the CI/CD pipeline)
2. Submitting to ENISA, CSIRTs, authorities or importers (a person does this by hand)
3. Deciding whether an incident is legally reportable (a human decision that ReleaseGate records)
4. Maintaining the vulnerability database (Dependency-Track does this)
5. The one-month final report for severe incidents (Art. 14(4)(c))

Limits: CycloneDX JSON is the only accepted format, Dependency-Track is the only CVE source, users are internal staff only, all times are UTC, and recall is tested only against the seeded dataset.

Q2.2 Included and excluded scope

1 Point

Grading comment:

(3-5 bullets each): State the capabilities every team will implement and the capabilities explicitly ***excluded***. Identify limits on supported data, users, formats, or operating conditions where they constrain the project. Reference the regulatory obligations the included capabilities support.

Included:

1. Release registration with a support-end date (REQ-01; Art. 13(8))
2. CycloneDX JSON SBOM upload, validation and rejection with a stated reason (REQ-02; Annex I, Part II(1))
3. CVE lookup grouped by support status, with an explicit message when the CVE is unknown or the source is unreachable (REQ-03, REQ-04; Annex I, Part II(1)–(2))
4. Incident cases with awareness time and classification, plus tracking of the 24h, 72h and 14-day deadlines (REQ-05, REQ-06; Art. 14(1)–(2), Art. 15)
5. Report data for each stage as JSON, and an append-only audit log (REQ-07, REQ-08; Art. 14(2), Art. 16)

Excluded:

1. Generating or attesting SBOMs (done by the CI/CD pipeline)
2. Submitting to ENISA, CSIRTs, authorities or importers (a person does this by hand)
3. Deciding whether an incident is legally reportable (a human decision that ReleaseGate records)
4. Maintaining the vulnerability database (Dependency-Track does this)
5. The one-month final report for severe incidents (Art. 14(4)(c))

Limits: CycloneDX JSON is the only accepted format, Dependency-Track is the only CVE source, users are internal staff only, all times are UTC, and recall is tested only against the seeded dataset.

Q2.3 Cost and feasibility estimate

1 Point

Grading comment:
Estimate the cost (number of person-hours) for each team to deliver this scope through Phase 6. Break the estimate into major work areas; include design, implementation, integration, validation, and deployment. State the available GenAI tools or licenses, expected use, and how they affect the estimate. Use ranges where uncertain; explain differences between teams.

1. **Design: 2 h per person (8 h team), week 1.** Data model, deadline rules and the report fields for each stage.
2. **Implementation: 5 h per person (20 h team), weeks 1–3.** REQ-01 to REQ-08.
3. **Integration: 2 h per person (8 h team), week 3.** Dependency-Track, the timer service and SBOM intake from the pipeline.
4. **Validation: 2 h per person (8 h team), week 4.** Recall on the seeded dataset, deadline edge cases, rejection reasons and log immutability.
5. **Deployment: 1 h per person (4 h team), week 4.** Environment setup and a short runbook.
6. **GenAI:** Every team is assumed to have a coding assistant (for example Claude Code or Copilot) that writes most of the boilerplate code and tests. Without it this scope would take roughly 3–4 times longer. People still write and check the deadline logic and the audit log.
7. **Risk:** 12 hours per person is tight. If a team falls behind, cut the integration work first: replace the external timer service with local persistent timers and use a local copy of the CVE data.

8. **Differences between teams:** Teams that already know CycloneDX or Dependency-Track will need fewer implementation hours. Teams new to timers or persistent storage should move hours from design into integration.

Q3.1

Q3.1 Requirements

REQ-01 — When a user registers a release with a product, version, and support-period end date, the system shall store it and treat it as supported until that date.
Source: Art. 13(8).

REQ-02 — When a CycloneDX JSON SBOM is submitted for a registered release, the system shall store its components against that release. The system shall reject invalid files with an error that states the reason.
Source: Annex I, Part II(1).

REQ-03 — When a user queries a Common Vulnerabilities and Exposures (CVE) identifier, the system shall return every release containing an affected component, grouped by support status, with 100% recall on the pod's seeded test dataset.
Source: Annex I, Part II(1)–(2).

REQ-04 — If the CVE is unknown or the vulnerability source is unreachable, the system shall say so explicitly rather than return an empty result.
Source: stakeholder need, since an empty result would give false assurance.

REQ-05 — When a user opens an incident case, the system shall record the awareness time (UTC), the user, and a mandatory or voluntary classification with a rationale.
Source: Art. 14(1); Art. 15.

REQ-06 — For a mandatory case, the system shall compute deadlines at awareness + 24 hours, awareness + 72 hours, and fix-available + 14 days. It shall show each deadline as pending, met, or overdue based on recorded submission times.
Source: Art. 14(2)(a)–(c).

REQ-07 — When a user requests report data for a stage, the system shall output JSON with that stage's fields (Section 5), listing any missing fields.
Source: Art. 14(2); Art. 16.

REQ-08 — The system shall log every change with its time and user, shall not allow log entries to be edited, and shall retain all data across restarts.
Source: Art. 14 evidence requirements.

Q3.2 -3.1_information.svg

3.1_behavior.svg

Q3.3

Five models were used, each chosen for the kind of information it needed to carry, with no requirement modeled twice. The release/SBOM data model (an ER diagram) covers REQ-01 and REQ-02 by fixing the storage shape — a release with at most one current SBOM listing many components — and deliberately omits a stored "supported" flag, since support status is a function of support_end_date and the current time and would go stale if persisted instead of recomputed on read. The SBOM submission workflow covers REQ-02's behavior separately from its data shape, because the requirement has a real branch a schema can't express: validation followed by either storage or rejection with a stated reason. The CVE query flowchart covers REQ-03 and REQ-04 together, since REQ-04 only exists as a modification of REQ-03's happy path, and models three terminal outcomes rather than one — a grouped result, an explicit "CVE unknown" notice, and an explicit "source unreachable" notice — so that neither failure mode can be implemented as a silent empty response. The incident/deadline state machine covers REQ-05 and REQ-06 as a case-level state (awareness, classification, rationale) containing three concurrent deadline clocks for mandatory cases, each independently pending, met, or overdue, which matches the requirement that the three CRA deadlines run in parallel off different anchor times (awareness vs. fix-available) rather than in sequence. Finally, the report-schema-and-invariants model covers REQ-07 and REQ-08 as structured examples rather than diagrams, since both requirements are best expressed as a concrete shape (a stage report JSON with an explicit missing_fields list) and a short list of invariants (append-only, timestamped, durable across restarts) that a diagram would only obscure.

Q4.1

AC-01 — Evaluates REQ-01. Given a fixed reference clock, an automated test submits four release registrations: one with a past support end date, one matching the current reference date, one with a future date, and one missing a support end date. The system passes if stored records classify the current and future releases as supported through the end of the boundary day, classify the past release as unsupported, and reject the incomplete input without saving; otherwise, it fails.

AC-02 — Evaluates REQ-02. Given two registered releases of a product, an automated test submits a valid 25-component CycloneDX JSON SBOM for the first release and queries both

inventories. The system passes if stored records return 100% of the 25 components with matching names and versions under the first release and zero components under the second; otherwise, it fails.

AC-03 — Evaluates REQ-02. Given a registered release, the evaluator submits malformed JSON, JSON missing required CycloneDX fields, plaintext, and a valid SBOM targeting an unregistered release. The system passes if automated checks confirm all four inputs are rejected with zero components stored, and human inspection verifies each error message states the specific failure reason; otherwise, it fails.

AC-04 — Evaluates REQ-03. Given the pod's seeded dataset of supported and unsupported releases with known vulnerable and patched components, an automated script queries five test CVEs. The system passes if every query completes within 10 seconds and matches the ground-truth manifest with 100% recall, 100% precision, and accurate support-status grouping; otherwise, it fails.

AC-05 — Evaluates REQ-04. Given the seeded dataset, an automated fault-injection test queries a valid CVE affecting no releases, an unknown CVE, and a valid CVE while the vulnerability source is unreachable. The system passes if the first returns zero affected releases while the second and third return explicit unknown-CVE and unreachable-source errors within 15 seconds rather than empty lists; otherwise, it fails.

AC-06 — Evaluates REQ-05. Given valid mandatory and voluntary incident submissions with UTC awareness times and rationales alongside an invalid submission missing a rationale or valid classification, the evaluator submits all three and reads back the stored records. The system passes if automated checks confirm the invalid request is rejected and valid cases preserve the user, classification, and UTC timestamp within plus or minus 2 seconds, and human inspection verifies the rationale text is intact; otherwise, it fails.

AC-07 — Evaluates REQ-06. Given a mandatory incident with fixed awareness and fix-available times, an automated clock-injection test checks initial deadlines, records an early warning at the exact 24-hour boundary, advances 1 second past the 72-hour deadline without a notification, and records a final report 13 days after the fix. The system passes if deadlines are computed at plus 24 hours, plus 72 hours, and plus 14 days with 0-second tolerance, and statuses transition from pending to met for the early warning and final report and overdue for the notification; otherwise, it fails.

AC-08 — Evaluates REQ-07. Given a complete mandatory incident and an incomplete incident missing two required 72-hour stage fields, the evaluator requests JSON reports for all stages. The system passes if automated schema checks and human inspection confirm all outputs are valid Section 5 JSON, with the complete case containing 100% of stage fields and an empty missing-fields list and the incomplete case listing the exact two missing fields; otherwise, it fails.

AC-09 — Evaluates REQ-08. Given an active system where a user registers a release, submits an SBOM, opens an incident, and records a submission, the evaluator queries the audit log and

attempts to edit or delete an entry. The system passes if automated tests confirm all four actions are logged chronologically with the user and UTC timestamp within plus or minus 2 seconds while rejecting edits, and code inspection confirms no update or delete paths exist; otherwise, it fails.

AC-10 — Evaluates REQ-01, REQ-02, REQ-05, REQ-06, and REQ-08. Given a system populated with releases, SBOM components, an active mandatory incident, and audit logs, an automated test captures a state snapshot, restarts the process, and re-queries all records. The system passes if the post-restart state matches the pre-restart snapshot with 100% data retention and 0-second timestamp drift; otherwise, it fails.

Q4.2

The pod will create a shared test dataset with sample product releases, valid CycloneDX JSON SBOMs, known vulnerability records, and mandatory and voluntary incident cases, plus invalid files, missing fields, unknown CVEs, and simulated service failures. Boundary cases will cover support expiration, submissions just before, at, and after deadlines, missing fix dates, and month and year changes. Before testing, both teams will check the expected affected releases, support status, deadlines, report fields, and error messages against the test data, Section 5 formats, and relevant CRA provisions. Tests will use the same saved vulnerability data, simulated service responses, and controlled UTC clock so results remain repeatable without relying on live services or AI-generated answers.

Q4.3

Another team can evaluate AC-01 through AC-04 and AC-06 through AC-08 entirely through the Section 5 web service interface without knowing our internal design by running automated HTTP checks against the shared dataset and controlled UTC clock, paired with human inspection of the returned JSON response artifacts to verify error reasons (AC-03), rationales (AC-06), and stage-report fields (AC-08). AC-05 and AC-10 also send requests and check outputs through the web interface, but require the evaluating team to use the test-runner environment to stop the mock vulnerability service for fault injection (AC-05) and to restart the application process before running an automated diff on pre- and post-restart JSON state snapshot artifacts (AC-10). Finally, while AC-09 uses the web interface to confirm that state changes are logged and edit requests fail, verifying true append-only immutability requires inspecting the team's database schema and persistence-layer source code artifacts to confirm that no update or delete operations exist on the audit store.

Q5

Define the common web-service interface that every team will provide. Automated tests must be able to send requests to any team's deployed service and check the observable results. A network-accessible implementation is required beginning with Phase 3.

Q5.1

Provide an OpenAPI document or an equivalent structured HTTP specification covering every operation needed for cross-team testing. For each operation, state its purpose and related requirement identifiers, HTTP method and path, request format, successful response, and error responses with status codes. Define relevant state changes and behavior needed to interpret results.

The pod will use a REST-style HTTP API with JSON request and response bodies. All timestamps are ISO 8601 UTC timestamps ending in Z, and all mutating application requests require an X-User-Id header so that changes can be attributed in the audit log. Authentication is outside the common testing scope; X-User-Id identifies the acting test user but is not an authentication mechanism. The interface exposes only behavior needed to test the common specification and does not constrain either team's internal database, framework, vulnerability scanner, workflow engine, or user interface.

Code is in txt file in the drive. Copy the code into the gradescope.
https://drive.google.com/drive/folders/1gXUPodcW2tnmmTfJfgLYfxJJPd4ndDIk?usp=sharing

Q5.2

Define the fields, types, required/optional status, and allowed values or constraints for the inputs and outputs. Specify units and formats where relevant. Include at least one valid request/response example for each core operation and one invalid-input example with its expected error. Cross-reference definitions in the contract rather than duplicating them.

General Data Rules:

-All request and response bodies use JSON unless otherwise stated.

-Dates use `YYYY-MM-DD`. Timestamps use ISO 8601 UTC format with a trailing Z (e.g., `2026-09-26T18:00:00Z`).

-System-generated identifiers (release_id, incident_id, event_id) are nonempty strings and should be treated as opaque values by clients.

-For state-changing operations, the request must contain the `X-User-Id` HTTP header (a nonempty string).

-A release is considered supported through the end of its `support_end_date` and becomes unsupported on the following UTC calendar day.

1. Register a release (POST /api/v1/releases)

-Input (ReleaseCreate schema):

- `-product` (string, required): Nonempty product name
- `-version` (string, required): Nonempty version identifier
- `-support_end_date` (string/date, required): YYYY-MM-DD

-Output (Release schema): Includes the input fields plus `release_id` (string, system-generated) and `support_status` (string: "supported" or "unsupported").

-Valid Request:

```
{
  "product": "ExampleRouter",
  "version": "2.4.1",
  "support_end_date": "2027-12-31"
}
```

-Valid Response (201 Created):

```
{
  "release_id": "rel-001",
  "product": "ExampleRouter",
  "version": "2.4.1",
  "support_end_date": "2027-12-31",
  "support_status": "supported"
}
```

-Invalid Request Example: A release request that omits the required `support_end_date`.

-Expected Error Response (400 Bad Request): The request must not create a release.

```
{
  "error": {
    "code": "INVALID_REQUEST",
    "message": "support_end_date is required"
  }
}
```


2. Submit an SBOM (PUT /api/v1/releases/{releaseId}/sbom)

-Constraints: The request must be a valid CycloneDX JSON document. Components are stored against the release identified by releaseId.

-Valid Request:


```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.6",
  "version": 1,
  "components": [
    {
      "type": "library",
      "name": "example-library",
      "version": "3.2.0",
      "purl": "pkg:npm/example-library@3.2.0"
    }
  ]
}
```


-**Valid Response (200 OK):**


```
{
  "release_id": "rel-001",
  "component_count": 1
}
```


3. Retrieve release components (GET /api/v1/releases/{releaseId}/components)

-Constraints: A registered release with no stored SBOM returns an empty components array. An unknown release returns 404.

-Valid Response (200 OK):


```
{
  "release_id": "rel-001",
```

```
  "components": [
    {
      "name": "example-library",
      "version": "3.2.0",
      "purl": "pkg:npm/example-library@3.2.0"
    }
  ]
}
```

4. Query affected releases (GET /api/v1/vulnerabilities/{cveId}/affected-releases)

-Constraints: `cveId` must follow the format CVE-YYYY-NNNN... A known CVE that affects no stored releases returns 200 with empty supported and unsupported arrays. An unknown CVE returns 404 (CVE_UNKNOWN). An unavailable vulnerability source returns 503.

-Valid Request: `GET /api/v1/vulnerabilities/CVE-2026-12345/affected-releases`

-Valid Response (200 OK):

```
{
  "cve_id": "CVE-2026-12345",
  "supported": [
    {
      "release_id": "rel-001",
      "product": "ExampleRouter",
      "version": "2.4.1",
      "support_end_date": "2027-12-31",
      "matched_components": [
        {
          "name": "example-library",
          "version": "3.2.0",
          "purl": "pkg:npm/example-library@3.2.0"
        }
      ]
    }
  ],
  "unsupported": []
}
```

5. Create an incident (POST /api/v1/incidents)

-Input (IncidentCreate schema):

- `cve_id` (string/null, optional): CVE-YYYY-NNNN... format

- `awareness_at` (string/datetime, required): ISO 8601 UTC

- `classification` (string, required): "mandatory" or "voluntary"

- `rationale` (string, required): Nonempty explanation

-Valid Request:

```
{
  "cve_id": "CVE-2026-12345",
  "awareness_at": "2026-09-26T18:00:00Z",
  "classification": "mandatory",
  "rationale": "Confirmed active exploitation affecting a supported release."
}
```

Valid Response (201 Created): (For mandatory cases, the first two deadlines are calculated as awareness_at + 24 hours and awareness_at + 72 hours).

```
{
  "incident_id": "inc-001",
  "cve_id": "CVE-2026-12345",
  "awareness_at": "2026-09-26T18:00:00Z",
  "created_by": "engineer-17",
  "classification": "mandatory",
  "rationale": "Confirmed active exploitation affecting a supported release.",
  "fix_available_at": null,
  "deadlines": {
    "early_warning": {
      "deadline_at": "2026-09-27T18:00:00Z",
      "submitted_at": null,
      "status": "pending"
    },
    "vulnerability_notification": {
      "deadline_at": "2026-09-29T18:00:00Z",
      "submitted_at": null,
      "status": "pending"
    },
    "final_report": {
      "deadline_at": null,
      "submitted_at": null,
      "status": "not_applicable"
    }
```

```
  }
}
```

6. Record corrective-measure availability (PATCH /api/v1/incidents/{incidentId})

-Valid Request: `{"fix_available_at": "2026-09-30T12:00:00Z"}`

-Valid Response (200 OK): Uses the existing Incident schema, but the final deadline updates to:

```
{
  "final_report": {
    "deadline_at": "2026-10-14T12:00:00Z",
    "submitted_at": null,
    "status": "pending"
  }
}
```

7. Record a report submission (POST /api/v1/incidents/{incidentId}/submissions)

-Valid Request: `{"stage": "early_warning", "submitted_at": "2026-09-27T17:45:00Z"}`

-Valid Response (201 Created): Uses the Incident schema. A submission at or before its deadline changes the status to "met":

```
{
  "early_warning": {
    "deadline_at": "2026-09-27T18:00:00Z",
    "submitted_at": "2026-09-27T17:45:00Z",
    "status": "met"
  }
}
```

8. Store and generate report data (PUT .../report-data/{stage} and GET .../reports/{stage})

-Constraints: Partial input is allowed because the system must identify required fields that are still missing.

-Valid Report-Data PUT Request:

```
{
  "product_information": "ExampleRouter 2.4.1",
```

```
  "exploit_nature": "Remote exploitation of vulnerable library",
  "vulnerability_nature": "Input validation vulnerability",
  "corrective_measures_taken": "Patched library version prepared",
  "user_mitigations": "Disable affected feature until update"
}
```


-Generated Report GET Response (200 OK):


```
{
  "incident_id": "inc-001",
  "stage": "vulnerability_notification",
  "generated_at": "2026-09-27T19:00:00Z",
  "data": {
    "product_information": "ExampleRouter 2.4.1",
    "exploit_nature": "Remote exploitation of vulnerable library",
    "vulnerability_nature": "Input validation vulnerability",
    "corrective_measures_taken": "Patched library version prepared",
    "user_mitigations": "Disable affected feature until update"
  },
  "missing_fields": []
}
```


9. Retrieve audit logs (GET /api/v1/audit-logs)

-Constraints: Returned chronologically. Attempts to modify or delete return 405 Method Not Allowed.

-Valid Response (200 OK):


```
{
  "events": [
    {
      "event_id": "evt-001",
      "timestamp": "2026-09-26T18:00:05Z",
      "user_id": "engineer-17",
      "action": "INCIDENT_CREATED",
      "entity_type": "incident",
      "entity_id": "inc-001",
      "details": {
        "classification": "mandatory"
      }
    }
  ]
```

}

Q5.3 Repeatable test setup

Grading comment:
(3-5 sentences): Explain how tests initialize or reset data, obtain any required authentication, and run against a selected team's base URL. Identify any shared fixtures, external-service assumptions, or ordering constraints. If the service is stateless or needs no authentication/reset, explicitly say so. Keep internal architectures and implementation technologies independent.

Before each automated test run, the evaluator will select the target team’s base URL and call POST /api/v1/test/reset to clear releases, SBOMs, incidents, report data, and audit logs so every test starts from the same state. No authentication is required for the common test interface; state-changing requests only include the required X-User-Id header so actions can be attributed in the audit log. Tests will use the pod’s shared fixtures, including the same sample releases, CycloneDX SBOMs, vulnerability records, incident cases, and controlled UTC clock, and may use the test vulnerability-source endpoint to simulate available or unavailable external vulnerability data. Tests that depend on existing state will create resources in the required order, such as registering a release before uploading its SBOM and creating an incident before recording submissions, while each team remains free to use any internal architecture or implementation technology.

Question 6.1: Phase 3

Capability to deliver: Release registration, CycloneDX SBOM ingestion, and baseline testability.

Artifacts to produce or update: Database schemas; endpoints for `/releases`, `/releases/{releaseId}/sbom`, `/test/reset`, and `/test/clock`.

Uncertainty to reduce: The technical risk of parsing nested CycloneDX JSON files and correctly normalizing them into relational tables without data loss.

Demonstration of completion: Successfully deploying the API to a network-accessible server, registering a release, and persisting the pod's shared test SBOM payload.

Automated validation to add: AC-01, AC-02, and AC-03 (validating REQ-01 and REQ-02).

Question 6.2: Phase 4

Capability to deliver: Vulnerability querying with fault injection and incident deadline tracking.

Artifacts to produce or update: Endpoints for `/vulnerabilities/{cveId}/affected-releases`, `/test/vulnerability-source`, and `/incidents`; implementation of temporal deadline math.

Uncertainty to reduce: The risk of timezone drift in UTC deadline computation and ensuring reliable behavior when external vulnerability sources are unreachable.

Demonstration of completion: Utilizing the mock test clock to open a mandatory incident, verifying computed 24h/72h deadlines, and querying a CVE to retrieve affected product versions.

Automated validation to add: AC-04, AC-05, AC-06, and AC-07 (validating REQ-03, REQ-04, REQ-05, and REQ-06).

Question 6.3: Phase 5

Capability to deliver: CRA compliance report generation, immutable audit logging, and cross-team test execution.

Artifacts to produce or update: Endpoints for `/incidents/{incidentId}/reports/{stage}` and `/audit-logs`; final cross-team testing scripts.

Uncertainty to reduce: Guaranteeing audit log immutability and ensuring complete data persistence across server process restarts.

Demonstration of completion: Retrieving a formatted JSON stage report and successfully running the automated test suite against podmates' live deployed API URLs.

Automated validation to add: AC-08, AC-09, and AC-10 (validating REQ-07 and REQ-08).

Question 6.4: Sequencing rationale

This order is highly feasible because it builds the system logically from the foundational data layer outward. We must establish reliable SBOM storage and release tracking (Phase 3) before we can build the CVE querying logic that relies on mapping those exact components (Phase 4). Similarly, the core incident state machine and deadline calculations (Phase 4) must function correctly before we can aggregate that state into compliance reports or test the immutable audit log (Phase 5). This sequencing reduces early technical risks by proving we can parse the massive CycloneDX file structure and inject a mock test clock before attempting complex timeline math. If progress falls behind, the pod would reduce scope by dropping the voluntary incident classification logic (REQ-05) to focus strictly on mandatory deadlines, or by stripping out the unreachable-source fault-injection (REQ-04, AC-05) to simplify the testing infrastructure.

Individual Group part

Q7.1 Architecture and data flow

draw.io in link

https://drive.google.com/file/d/10C5LDNyufaXbU8mWUg1eDbgqB83VN9iG/view?usp=sharing

Q7.2 Design decisions

Decision 1: Reuse Dependency-Track for matching, with our own release registry in front of it.
We will map each registered release (REQ-01) to a Dependency-Track project version and upload its CycloneDX SBOM [8] there, while keeping the authoritative release, support-end, and component records in our own PostgreSQL database [6]. The alternative was a custom matcher that queries OSV [9] directly against our component table. Dependency-Track gives us mature SBOM-based component analysis through an API-first design [3], supporting REQ-03's 100% recall target and Annex I, Part II(1) [1]. The cost is a second datastore we must keep in sync, plus an extra network hop that affects AC-04's 10-second budget. It also means "source unreachable" (REQ-04, AC-05) is defined as Dependency-Track being unreachable, so we must detect that explicitly rather than letting a failed call return an empty list.

Decision 2: Store incident state and compute deadlines on read in PostgreSQL, instead of using TheHive or a workflow engine.

Phase 1 proposed TheHive [4] for case management, and we also considered a durable workflow engine such as Temporal [7] for timers. We are instead storing incidents, awareness time, fix-available time, and submission times as PostgreSQL rows [6], and deriving each deadline's pending, met, or overdue status from those timestamps and the injectable clock on every read. Running timers can drift or be lost on a crash, while stored timestamps survive restarts by construction. This directly serves REQ-06's Article 14(2)(a)–(c) deadlines [1], REQ-08, AC-07's 0-second tolerance, and AC-10. The tradeoff is losing TheHive's case UI, task assignment, and evidence attachments, and having no proactive reminders unless we later add a polling job.

Q7.3 Operation and observability

We will deploy the API as a Docker Compose stack on [host]. It has three containers: our [framework] service, PostgreSQL [6] on a named persistent volume, and Dependency-Track backed by its own Postgres volume, as its documentation recommends for production [2]. The host must meet Dependency-Track's minimum of 2 GB RAM and 2 CPU cores, and ideally the recommended 8 GB and 4 cores [2]. Nightly pg_dump backups protect the data. Secrets, including the database password and the Dependency-Track API key, live in an untracked .env file on the host, and only our API port is publicly exposed. For failure detection, a /health endpoint checks database and Dependency-Track connectivity, and an external uptime ping hits it. Structured JSON logs record the request ID, X-User-Id, endpoint, status, latency, and any Dependency-Track errors or timeouts. We will track per-endpoint p95 latency against the AC-04 and AC-05 limits, the Dependency-Track failure rate, SBOM rejection counts by reason, and the number of overdue deadlines. Together with the append-only audit log, these signals let us reconstruct which input, user, and clock value produced an incorrect result.

Q7.4 Validation strategy

Our first prototype will ingest the pod's shared 25-component CycloneDX SBOM [8] into both our database and Dependency-Track [3], then query one seeded CVE. This exercises the riskiest integration path (AC-02, AC-04) before incident handling is built. Static checks will run on every push in GitHub Actions: linting, type checking, and validation of responses against the Section 5 OpenAPI contract. Dynamic tests will also run in CI and block merges on failure. They include unit tests for the Article 14(2) deadline math [1] using the injected clock, covering boundary and month/year rollover cases, and the AC-01 through AC-10 HTTP suite against a freshly reset Compose stack with a mocked vulnerability source. We will investigate a failure by replaying its logged request ID and clock value, and every reproduced bug becomes a permanent regression test.

Q8.1 Capability decisions

SBOM storage and vulnerability matching: OWASP Dependency-Track [2], [3]

- Why it fits: It ingests CycloneDX [8] SBOMs natively and exposes an API-first design with OpenAPI documentation [3]. This supports REQ-03 and Annex I, Part II(1)–(2) [1] without building a scanner ourselves.
- Integration work: Map each release to a project version, upload SBOMs through the REST API, and query the projects affected by a vulnerability. We must also translate timeouts and unknown identifiers into REQ-04 errors.
- Cost and licensing: It is open source and free to use [3].
- Reliability and resources: It needs at least 2 GB RAM and 2 cores, with 8 GB and 4 cores recommended, and a separate database in production [2]. It is a single point of failure for REQ-03.
- Maintenance: Pin the image version and keep its vulnerability mirror synced.

Persistence and deadline tracking: PostgreSQL [6]

- Why it fits: It is free under the PostgreSQL License and ACID-durable across restarts, which covers AC-10.
- Security: We enforce the append-only audit log (REQ-08, AC-09) by granting the application role only INSERT and SELECT on the audit table.
- Alternative rejected: We chose it over Temporal [7] because deadlines computed on read don't need workflow durability, and it avoids operating an extra cluster.

Incident case management: TheHive, not adopted for the common scope [4]

- Licensing: TheHive requires a license. The Community tier is free but must be requested, while Gold and Platinum are paid and obtained through StrangeBee [4].
- Why rejected: Adopting it adds licensing overhead and another platform to operate, and it doesn't implement the Article 14(2) [1] deadline logic, which we would have to build anyway.

SBOM generation (optional CI demo, outside the common API): Syft [5]
Syft is released under Apache-2.0 and emits CycloneDX JSON [5]. REQ-02 accepts submitted SBOMs, so Syft is only a convenience for our own pipeline.

GenAI
We have ChatGPT Plus, Gemini Pro, and Claude Pro per member. We would use them for boilerplate and test scaffolding.

Q8.2 GenAI Use

Our team will use ChatGPT Plus, Gemini Pro, and Claude Pro for development assistance, mainly for boilerplate code, test scaffolding, debugging, and documentation; GenAI is not part of the running system. AI-generated code and tests will be reviewed by team members and validated through our static checks, automated tests, and the shared AC-01 through AC-10 test suite before being accepted. We will not provide production data, credentials, or secrets to GenAI tools, and all regulatory deadline logic and audit-log behavior will be verified against the specification rather than trusting AI output. GenAI may reduce implementation time, but it adds no runtime cost or dependency and does not affect the deployed system's reliability.

Q9.1 Priority risks

Risk 1 (highest): Dependency-Track can't meet REQ-03/AC-04 or support REQ-04 behavior.

- Uncertainty: Its matching [3] may miss seeded components, for example because of missing PURLs or unusual version ranges. At the minimum resource tier [2], its API may exceed the 10-second limit. It may also fail to distinguish "unknown CVE" from "no affected releases."
- Effect: We would fail AC-04 and AC-05, and the Phase 4 capability would slip.
- Investigation: [Name] will run the shared SBOMs [8] and 5 seeded CVEs through Dependency-Track, measuring recall, precision, and p95 latency against the ground-truth manifest.
- Deadline: Must be resolved by the start of Phase 4 (Section 6.2).
- Evidence for the decision: 100% recall on the seed set, p95 latency under 10 s, and a distinguishable unknown-CVE response.
- Fallback: Do our own matching in PostgreSQL [6] against a saved local copy of OSV records [9], which also simplifies REQ-04 fault injection.

Risk 2: Ambiguity over when "awareness" starts the Article 14 clock.

- Uncertainty: Article 14(2) measures deadlines from when the manufacturer becomes aware of an actively exploited vulnerability [1]. The pod hasn't agreed whether that means the raw alert time or engineer-verified confirmation, which was our open Phase 1 disagreement.
- Effect: AC-06 and AC-07 would produce conflicting expected deadlines across teams.
- Investigation: [Name] will review Article 14 [1] and any ENISA or Commission guidance on awareness, then propose a rule to the pod.
- Deadline: Must be resolved before the shared dataset is frozen in Phase 3 (Section 6.1).
- Evidence for the decision: Cited guidance text and a pod-agreed rule.
- Fallback: Treat awareness_time as a user-supplied field, as REQ-05 already does, so the system never infers it and tests stay deterministic regardless of the legal interpretation.

Q9.2 Risks requiring pod agreement

Risk 2 requires pod agreement because different interpretations of when Article 14 “awareness” begins would change the expected deadlines in AC-06 and AC-07; this must be resolved before the shared Phase 3 dataset is frozen. The pod should either agree on a common interpretation or keep `awareness_time` as a user-supplied field so both teams compute identical deadlines from the same input. Risk 1 does not currently require a shared-scope change, but if Dependency-Track cannot satisfy REQ-03/REQ-04 and AC-04/AC-05, the pod must decide before Phase 4 whether the common specification or testing assumptions need to change.

References

[1] European Parliament and Council of the European Union. "Regulation (EU) 2024/2847 (Cyber Resilience Act)." October 23, 2024. https://eur-lex.europa.eu/eli/reg/2024/2847/oj. Accessed: 2026-09-26.

[2] OWASP Dependency-Track. "Deploying Docker Container." Dependency-Track Documentation, v4.14. https://docs.dependencytrack.org/getting-started/deploy-docker/. Accessed: 2026-09-26.

[3] OWASP Dependency-Track. "Dependency-Track." GitHub repository. https://github.com/DependencyTrack/dependency-track. Accessed: 2026-09-26.

[4] StrangeBee. "TheHive Licenses." TheHive Documentation. https://docs.strangebee.com/thehive/installation/licenses/about-licenses/. Accessed: 2026-09-26.

[5] Anchore. "Syft." GitHub repository. https://github.com/anchore/syft. Accessed: 2026-09-26.

[6] The PostgreSQL Global Development Group. "PostgreSQL License." https://www.postgresql.org/about/licence/. Accessed: 2026-09-26.

[7] Temporal Technologies. "Temporal." GitHub repository. https://github.com/temporalio/temporal. Accessed: 2026-09-26.

[8] OWASP CycloneDX. "CycloneDX Specification Overview." https://cyclonedx.org/specification/overview/. Accessed: 2026-09-26.

[9] Google Open Source Security Team. "OSV: Open Source Vulnerabilities." https://osv.dev/. Accessed: 2026-09-26.